



miw.etsisi.upm.es • 2 Index	
• Introduction ◦ Basic concepts ◦ Javascript usage • Javascript and ECMAScript ◦ Variables and operators ◦ Control sentences ◦ Arrow functions ◦ User objects • Definition: properties and methods • Prototypes • ECMA 6: Objects and classes	• Language objects ◦ Arrays ◦ Number, Boolean and Math ◦ Date and String ◦ Map and Set • Events ◦ Event management ◦ Advanced treatment: event • Browser objects ◦ Form elements ◦ Document ◦ Windows • DOM manipulation • Other characteristics and methods

Introduction to Javascript

miw.etsisi.upm.es • 3

- It allows to introduce dynamism in the html pages increasing its capacities.
 - Perform arithmetic calculations.
 - Move, hide and show visual elements.
 - Validate the information contained in forms.
 - Dynamically generate html pages.
 - Interact with Active-X objects, Java,...
 - Application state maintenance
- JavaScript evolved from LiveScript (1996 - Netscape - Sun Microsystems) to version 5, standardized in 2011 as ECMAScript v5 (European Computer Manufacturers Association).
- It is the ISO 16262 standard
- **Version 6** was standardized in June 2015
- Browsers do not yet have full ECMA6 support
- There are compilers ("transpilers": translate + compiler) that, based on a library, translate the code so that it works with all browsers (ex: Babel)

Introduction to Javascript

miw.etsisi.upm.es • 4

- Embedded in HTML.
- It allows the treatment of events.
- Interpreted by the browser.
- Improves the performance of web applications by offloading part of the process to the client.
- The language is object-oriented, prototype-based, imperative, untyped, and dynamic.
- There is a version for server processing (Nodejs)
- Newer versions tend to formalize typing and dynamic features which can lead to bugs when programming

miw.etsisi.upm.es • 5

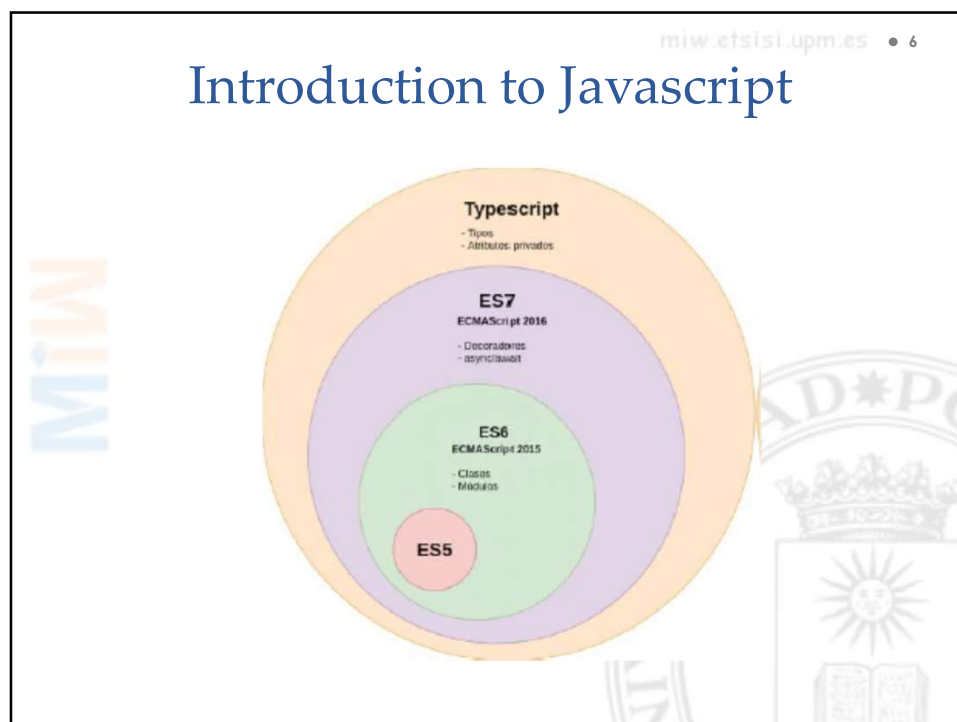
Introduction to Javascript

W: ECMAScript - Wikipedia, la enciclopedia libre

https://es.wikipedia.org/wiki/ECMAScript#ES2020

2	Junio de 1998	Cambios editoriales para mantener la especificación completa alineada con el estándar internacional ISO/IEC 16262	Mike Cowlishaw
3	Diciembre de 1999	Se agregaron expresiones regulares, mejor manejo de strings, nuevo control de declaraciones, manejo de excepciones con try/catch, definición más estricta de errores, formato para la salida numérica y otras mejoras.	Mike Cowlishaw
4	Abandonado	La cuarta edición fue abandonada debido a diferencias políticas respecto a la complejidad del lenguaje. Muchas características propuestas para la cuarta edición fueron completamente abandonadas; algunas fueron propuestas para la edición ECMAScript Harmony.	
5	Diciembre de 2009	Agrega el modo estricto ("strict mode"), un subconjunto destinado a proporcionar una mejor comprobación de errores y evitar constructores propensos a errores. Aclara varias ambigüedades de la tercera edición, y afina el comportamiento de las implementaciones del "mundo real" que difieren consistentemente desde esa especificación. Agrega algunas nuevas características, como getters y setters, librería para el soporte de JSON, y una más completa reflexión sobre las propiedades de los objetos. ¹¹	Pratap Lakshman, Allen Wirfs-Brock
5.1	Junio de 2011	Esta edición 5.1 de la ECMAScript Standard está completamente alineada con la tercera edición del estándar internacional ISO/IEC 16262:2011.	Pratap Lakshman, Allen Wirfs-Brock
6	Junio de 2015 ¹²	La sexta edición agrega cambios significativos en la sintaxis para escribir aplicaciones complejas, incluyendo clases y módulos, definiéndolos semánticamente en los mismos términos del modo estricto de la edición ECMAScript 5. Otras nuevas características incluyen iteradores <i>for...of</i> loops, generadores y generador de expresiones estilo Python, funciones de dirección, datos binarios, colecciones (mapas, sets, mapas débiles), y proxies (metaprogramación para objetos virtuales y wrappers). Al ser la primera especificación "ECMAScript Harmony", es también conocida como "ES6 Harmony".	Allen Wirfs-Brock
7	Junio de 2016	La séptima edición fue una mera actualización de la versión 6. Incorpora el método <code>Array.prototype.includes()</code> y el operador exponencial (<code>**</code>).	Brian Terison
8	Junio de 2017	La 8ª edición, oficialmente conocida como ECMAScript 2017, fue finalizada en junio de 2017 ¹¹ . Incluye constructores <code>async/await</code> , los cuales funcionan usando generadores y promesas.	Brian Terison
9	Junio de 2018	La 9ª edición, oficialmente conocida como ECMAScript 2018, incluye operadores <code>rest/spread</code> para variables (tres puntos: <code>...identificador</code>), iteración asíncronica, <code>Promise.prototype.finally()</code>	Brian Terison
10	Enero de 2019	La 10ª edición, oficialmente conocida como ECMAScript 2019, incorporó <code>Array.flat()</code> , <code>Array.flatMap()</code> , <code>String.trimStart()</code> , <code>String.trimEnd()</code> , errores opcionales en el bloque <code>catch</code> , <code>Object.fromEntries()</code> , <code>Symbol.description</code>	Mathias Bynens
11	Junio 2020	Ver 11ª edición – ECMAScript 2020	Jordan Harband, Kevin

https://es.wikipedia.org/wiki/ECMAScript#ES2020



JavaScript embedded in HTML

- JavaScript may appear in four different ways:

- Between the `<script>` and `</script>` tags

```
<script type="application/javascript">
  JavaScript Code
</script>
```

- Importing javascript code from an external file.

```
<script src="library.js"></script>
```

- As a response or treatment of an event

```
<input type="button" value="botón" onclick="JavaScript code" />
```

- As a resource to get when clicking a link.

```
<a href="javascript:JavaScript code">enlace</a>
```

Javascript basics

- JavaScript execution can be disabled by the user in his browser
- Instructions in a sequence are separated by `;` or carriage returns and are executed from top to bottom.

```
statement 1;
instruction 2;
.....
instruction N;
```

- It is recommended to always use `;` by analogy with other languages and allow the code to be compressed so that it takes up less space.

- Code comments:

- Comments until end of line `// ...`
`// This is a comment until the end of the line`
- Multi-line comments `/*...*/`
`/* This is a comment with multiple lines */`

ECMA 6: Constants

- To define a constant, simply prepend the reserved word `const` to an assignment of a value to a variable.

```
const PI = 3.14;
const DATE_SEPARATOR = "-";
```

- They cannot be re-declared nor change their value by reassignment
- Follow the same scoping rules as variables
- Warning: if the assigned value is an object, the object can change, although the constant cannot.
- As usual, the identifier is usually written in uppercase

variables declaration

- JavaScript is not typed and is also dynamic (it is not necessary to declare the variables and their type can change during execution)
- If it is done (recommended), they are declared using the word `var`.
- `var name = "Santiago";`
- Before using a variable in JavaScript it is not necessary to declare it.
- In the declaration the variables can be initialized by assigning them a value by means of the operator `=` (`variablename = value`).
- Identifiers in JavaScript:
 - First character must be (a-z or A-Z) or underscore (`_`) or dollar (`$`)
 - Other characters (a-z or A-Z or 0-9 or `_`)
 - They cannot have blank spaces
 - They cannot match reserved words (`var`, `function`, `for`, `if`, ...)
 - It is **case sensitive**
- There is automatic conversion of types according to the operations used to handle the data.

ECMA6: block scope variables

- Block-scoped variables can be defined, which avoid the so-called "hoisting" effect by using the `let` reserved word

```
for (let i = 0; i < 5; i++){
  let x = 2 * i;
  console.log(i);
  console.log (x);
}
console.log(x); // Error: x is not defined
```

- Defined in a block, the scope will be the block
- Defined in a function, the scope will be the function
- Defined in a script, the scope will be that script
- Today there is a tendency to use `let` instead of `var`

Variables types

- JavaScript has the following types of variables:
 - Numerical: integers or real.
 - Character strings: contain sequences of characters. They can be enclosed in single or double quotes.
 - Boolean: only two `true` or `false` values.
 - null: variable that has been assigned the value `null`
 - undefined: variable value declared but having received a value through an assignment
 - Objects: contain references to an object.
- JavaScript also has language-specific objects for numbers, strings, and booleans.
- A variable that contains a number, string or boolean can use the methods of the corresponding objects Eg:
`"Hello".toUpperCase();`

ECMA6: String interpolation

- You can interpolate values of expressions that are evaluated if they are enclosed in **backticks**. To do this, use **\$** and enclose the expression in braces (**\${}**)

```
var nombre = "Santiago";
var laCadena = `El nombre es ${nombre}`;
console.log (laCadena);
```

- You may use strings for more than one line

```
var nombre = "Santiago";
console.log (laCadena);
var mensaje = `Hola ${nombre}
bienvenido al sistema`;
console.log(mensaje);
```

Arithmetic operators

- They are used to do basic arithmetic calculations.

◦ Sum	+
◦ Subtraction	-
◦ Multiplication	*
◦ Division	/
◦ Remainder of integer division	%
◦ Incrementing a variable	++
◦ Decrement a variable	--

```
variable = A + 3 * 2;
variable ++;
variable= -- B % 2
```

String operators

- It is used in string manipulation: string concatenation **+**

```
greeting = "Good morning" + 'Afternoon';
Variable = greeting + ", students";
```

- Remark: If an operator is a string, the **+** operator will concatenate the text

```
a = "1" + 2 // a contains "12"
```

Relation operators

- They compare two values, returning **true** or **false** depending on whether the indicated relationship is fulfilled.
 - Equal-to operator `==`
 - Not-equal operator `!=`
 - greater-than operator `>`
 - Greater-or-equal-to operator `>=`
 - Less-than operator `<`
 - Less-or-equal-to operator `<=`
 - Identity operator `===`
 - Non-identity operator `!==`

```
minor_age = age < 18;
found = key == 'PeTeTe';
```

- If both operators are **objects**, the operators compare the references of the objects.

Logical operators

- They are applied on boolean operands.
 - operator not `!`
 - operator and `&&`
 - or operator `||`

```
young = (age >= 18) && (age <= 30);
correct = !(a < b) || (c == d)
```

Bit operators

- They operate with numbers in their binary representation (32 bits).
 - Complement on one operator `~`
 - Bit operator AND `&`
 - Bit's OR operator `|`
 - Bit operator "exclusive-or" `^`
 - Left shift operator `<<`
 - Right shift operator with sign propagation: `>>`
 - Right shift operator with padding of 0's `>>>`

```
var Mi_Num = 5;
var res = Mi_Num >>> 1 // will be 2
```

assignment and arithmetics operators

- | | |
|--------------------------|---------------------|
| ▪ <code>x = x + y</code> | <code>x += y</code> |
| ▪ <code>x = x - y</code> | <code>x -= y</code> |
| ▪ <code>x = x * y</code> | <code>x *= y</code> |
| ▪ <code>x = x / y</code> | <code>x /= y</code> |
| ▪ <code>x = x % y</code> | <code>x %= y</code> |

Operators precedence

- Operator precedence, from low to highest is:

Assignment	= += -= *= /= %=
Conditional	<<= >>= >>>= &= ^= =
"or"	?:
"and"	
"or" (bit)	&&
"xor" (bit)	
"and" (bit)	^
Equal	&
Comparison	== !=
Displacement	< <= > >=
Addition/Subtraction	<< >> >>>
Multiplication/Division	+ -
Denial/Increase	* /
	! ~ - ++ --

Using `()` the precedence may vary.

Conditional operator

- Conditional operator syntax is:

```
if (conditional expression) {
    javascript code if expression is true
}
else {
    javascript code if expression is false
}
```

- The `if` statement may not have an `else` part.
- Nested `if` statements allow you to place one `if` statement inside another.
- It is not necessary to write the braces (both in the `if` part and in the `else` part) when the code consists of a single statement.
- The following conditional expression:

```
if (condition)
    variable = Value1;
else
    variable = Value2;
```

Is equivalent to

```
variable = (condition) ? Value1 : Value2;
```

For statement

- For loop:

```
for (initial expression ;
    conditional expression ;
    update expression)
{
  javascript code that will be executed
}
```

initial expression: initialization of the conditions or government variable of the loop

conditional expression : continuation condition in the loop.

update expression: update the conditions or variables of the loop.

for .. in loop

- A for .. in loop accesses each property of an object (or each element of an array)

```
for (property in myObject){
  //elements processing
  myObject[property]
}
```

- The for in loop can access properties or methods that do not belong to the object but to its prototype
- To avoid this, the `hasOwnProperty` method must be used, which will be true if the property belongs to the object and not to its prototype.

```
for (property in myObject){
  if (miObjeto.hasOwnProperty(property))
    //elements processing myObject[property]
}
```

ECMA 6: for ... of loop

- The for ... of loop creates a loop over iterable objects (Array, Map, Set, String, arguments, with an iteration over each of its elements

```
for (element of iterableObject){
    //element processing
}
```

- Example:

```
let myArray = [1,3,5,7,9];
let result = 0;
for (let element of myArray){
    result = result + element;
}
console.log (result); // === 25
```

while and do ... while loops

- while:

```
while (conditional expression){
    javascript code that will be executed
}
```

- do ... while:

```
do {
    javascript code that will be executed
}while (conditional expression)
```

Loop breaking

- A loop can be stopped from within with the **break** statement. This statement causes the next line of code after the loop to be executed.
- **break** can be used with tags to indicate the part of the loop to break.
- The **continue** statement is a way to control a loop so that it restarts the process immediately before the beginning of the loop and starts the next cycle.
- **continue** returns to the evaluation of the condition in a **while** loop or to the update expression of a **for** loop

Switch sentence

- Its syntax is:

```
switch (expression) {
  case value1:
    sentence1;
    break;
  case value:
    sentence2;
    break;
  ...
  default: default sentence;
}
```

- **switch** evaluates the expression and then compares the result with the labels that are defined in the **case** part.
- If there is a **break**, it causes the program to jump to the next statement after the **switch**.
- If it does not find any similarity, it executes the statement that exists in the **default** tag.

Exception management

- ◉ An exception is a runtime error. They are handled by the statement:

```
try{
    statements that are executed normally. if an
    exception occurs in one, it jumps to the catch
    statements
}
catch (e){
    exception handling statements
}
finally{
    statements that are always executed after the
    previous ones
}
```

- ◉ Exceptions can be thrown using the **throw** statement.

```
throw "exception"
throw new Error("exception");
```

Exception management example

```
function fact(n){
    if (n < 0) throw "negative number" //new Error("negative number")
    else
        if (n == 0)
            return 1
        else return n*fact(n-1);
}

x = 3;
try{
    x = fact(x);
}
catch (e) {
    x = Number.NEGATIVE_INFINITY
    alert(e);
}
finally {alert(x);}
```

Function definition

- **Declarative function**. The function is a statement by itself that begins with the keyword `function` followed by the name of the function, the parameters in parentheses, and code that implements the function. These definitions are parsed only once.

```
function name(parameters) {
    function code
}
```

- **Anonymous function**. They are parsed only once. It is assigned to a variable and has no name. The format of the definition is:

```
var func = function (parameters) {
    function code
}
```

- **Literal Function or expression function**. They are created via the Function constructor. They are parsed each time they are accessed and do not have a specific name

```
var variable = new Function("param1",...,"paramN","Function code");
```

Functions in JavaScript are objects so a function can have properties and methods!!!

functions definition (ii)

- A function returns the value that is specified in the `return` statement (string, number, boolean, or object).
- A function can use `multiple return` statements if required by the program, although there should ONLY be one for each group of statements.
- Functions are normally called by their identifier passing the necessary parameters.

- **Declarative function**

```
function add1(x,y) {
    return x + y;
}
```

- **Anonymous function**

```
var add2 = function (x,y) {
    return x + y;
}
```

- **Literal function or expression function.**

```
var add3 = new Function("x","y",
    "return x + y");
```

Parameters passed to functions

- Parameters are passed:
- By **value**: Copy of string, number and boolean data types is passed.
- By **reference**: Objects are passed a reference, not a copy.
- The **arguments** array contains all the parameters received by the function

```
function genericAddition() {
    var acumulador = 0;
    for (var i=0; i<arguments.length;i++)
        acumulador += arguments[i]
    return acumulador;
}
```

Parameters passed to functions

- A function called with more parameters than defined ignores the remaining parameters.
- A function called with fewer parameters than defined works with undefined values for the missing parameters.
- Since functions are objects, one function can be passed as a parameter to another function

```
function parameter(x, y){return (x*y)}

function process(vector, value, myfunction){
    for (var i=0; i < vector.length; i++)
        vector[i] = myfunction(vector[i],value);
}

vector = new Array(1, 2, 3);
process(vector, 10, parameter);
```

ECMA 6: parameter default values

- Default values can be defined for function parameters:

```
function aFunction(a, b = 5, c = 12){
    return a + b + c;
}
```

The function call: `aFunction(3); // === 20`

- Passing a value overrides the default value:

```
aFunction(3,1,1) // === 5
```

ECMA6: “rest” parameters

- The “rest” parameters allow the representation as an array of an indefinite number of parameters in a function

```
function (param1, param2, ...restParam) {
  console.log ("There are: " + restParam.length);
}
```

- It only affects those parameters that do not have an identifier

```
function f (x, y, ...a) {
  return (x + y) * a.length
}
alert (f(1,2,3,4,5,6,7)); // === 15 (1+2)*5
```

- They can be disintegrated in the header

```
function (...[p1, p2, p3]) {
  return p1*p2*p3;
}
```

ECMA6: Disaggregation of parameters

- The parameters passed as arrays or objects can be disaggregated in the declaration of the headers

```
function f1 ([ elem1, elem2 ]) {
  console.log(elem1, elem2)
}
```

```
function f2 ({ prop1: valor1, prop2: valor2 }) {
  console.log(valor1, valor2)
}
```

```
function f3 ({ prop1, prop2 }) {
  console.log(prop1, prop2)
}
```

```
f1([ "Santiago", 45 ]);
f2({ prop1: "Santiago", prop2: 45 });
f3({ prop1: "Santiago", prop2: 45 });
```


ECMA6: arrow functions

- A new syntax can be used for defining functions called “**arrow**” functions.

```
var teachers = ['Santiago', 'Pedro', 'Manuel'];

teachers.forEach(function(myvalue) {
  console.log (myvalue + " is a
teacher");
});
```

Is equivalent to:

```
profesores.forEach(value => {
  console.log (value + " is a teacher");
});
```

- Arrow functions are always anonymous

ECMA6: arrow functions (ii)

- If more than one parameter is needed:

```
var teachers = ['Santiago', 'Pedro', 'Manuel'];

teachers.forEach(function(value, index){
  console.log ("Position %s : %s", index, value);
} );

teachers.forEach( (value,index) => {
  console.log ("Position %s : %s", index, value);
} );
```

- If access to *arguments* array is needed, arrow functions won't work

ECMA6: arrow functions and *this*

- In ECMAScript v5, *this* refers to the current context

```
function Counter() {
  this.times = 0;

  setInterval(function mycount() {
    this.times++; // error
    console.log(this.times);
  }, 1000);
}
```

mycounter = new Counter();
It is solved using another variable:

```
function Counter() {
  var self = this;
  self.times = 0;

  setInterval(function mycount() {
    self.times++;
    console.log(self.times);
  }, 1000);
}

mycounter = new Contador();
```

ECMA6: arrow functions and *this* (ii)

- In ECMAScript v6, in arrow functions, *this* refers to its container:

```
function Counter() {
  this.times = 0;

  setInterval(() => {
    this.times++;
    console.log(this.times);
  }, 1000);
}

mycounter = new Contador();
```

ECMA6: “generator” functions

- The "generator" functions are functions that allow reentry into them preserving the context (variables)

```
function* counter(){
  let times = 0;
  while (times < 10){
    times++;
    yield times;
  }
}

var mycounter = counter();
console.log(mycounter.next().value); //1
console.log(mycounter.next().value); //2
console.log(mycounter.next().value); //3
```

- The function is defined with **function***
- **yield** stops execution and sets the return value
- The created object is iterable (has **next()** and **value**)

Some “functions” (window methods)

- **isNaN(value)** function that checks if the value is **NaN**. Returns **true** if it is not a number and **false** when it is.
- **parseInt(string, base)**, examines the string and tries to convert it to a number in the base specified as a parameter. This function returns NaN (Not a Number) if it can't get a number
- **parseFloat(string)** same as **parseInt** but with real numbers
- **isFinite()** returns **true** if the value is not infinity or **NaN**
- **setInterval()** periodically executes an action after the milliseconds it receives as a parameter has elapsed.
setInterval(callback, milliseconds)
- **setTimeout()** executes (just once) an action after the milliseconds it receives as a parameter has elapsed.
setTimeout(callback, milliseconds)

Some “functions” (window methods) ii

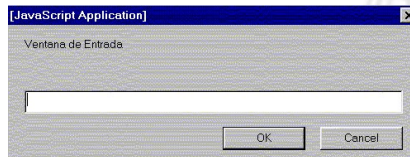
```
alert('Esta es una ventana de Alerta');
```



```
conf = confirm("Ventana de Confirmación");
```



```
entrada = prompt("Ventana de Entrada", "");
```



Language Objects

- The primitive types **string**, **number** and **boolean** have their correspondence with objects. When a method is called on a primitive type, JavaScript treats it as if it were an object.
- Built-in Objects:
 - **Array()**
 - **Boolean()**
 - **Date()**
 - **Math()**
 - **Number()**
 - **String()**
 - **RegExp()**

ECMA 6

- **Set()**
- **Map()**

Array Object

- To use it, just create an instance of the array object and assign values to it. The object is called `Array`.
- To create instances, the `new` keyword is used.
- Ex: `var vector = new Array()`
- The `Array` object starts at index 0.
- `var info = new Array(3)` creates elements with index values 0, 1, 2.
- `var info = new Array("one",2)` creates a 2-element array containing the values "one" and 2.
- An array literal can also be created by enclosing a list of values in square brackets.

Ex: `var vector = ["one", 2]`
`var othervector = [];`

- The `length` property indicates how many elements the array instance has.
- The array grows automatically as new positions are needed.

Array Object

- `delete a[i]` assigns the value `null` to the element located at position `i` of the vector
- Arrays of more than one dimension can be defined.
- To define them, simply create arrays for each of the dimensions of the array.

```
var f1 = new Array(00, 01)
var f2 = new Array(10, 11)
var f3 = new Array(20, 21)
var data = new Array (f1,f2,f3)
```

00	01
10	11
20	21

- Accessing a two-dimensional array is done as follows:
`variablename[row][column]`

Array Object

- `concat(array1, array2,...)` returns the result of concatenating the array with `array1, array2,...`
- `join(separator)` returns a String with all the elements of the vector separated by the String separator
- `pop()` removes the last element of the array and returns it
- `push(elem1, elem2,...)` adds the elements `elem1, elem2,...` to the end of the array and returns the new size
- `reverse()` reverses the order of the array elements
- `shift()` removes the first element of the array and returns it
- `slice(start, end)` returns the portion of the array between start and end. If beginning is negative, it is counted from the end of the array.
- `sort(criterion)` sorts the elements of the array. Optionally, you can indicate an ordering `criterion`
- `splice(index, quantity, elem1, elem2,...)` Removes from the array as many elements as indicated by `quantity` from `index` and replaces them with `elem1, elem2,...`
- To delete an element: `myarray.splice(position,1)`
- `unshift((elem1, elem2,...)` adds the elements `elem1, elem2,...` to the beginning of the array and returns the new size
- `forEach(function (element, index){`
 `// element treatment }`
 execute the callback function for each element of the array

Boolean Object

- ◎ Used to convert non-boolean values to Booleans
- ◎ Values:
 - `var variable= new Boolean(string)`
 - `Boolean()` -> returns false
 - `Boolean(string)` if string is : 0, nulls, false or text without quotes or empty quotes -> returns false
 - `Boolean(string)` if string is : true or text with quotes -> returns true

Number Object

Static properties:

- `Number.MAX_VALUE` largest positive number in Javascript
- `Number.MIN_VALUE` minor positive number in Javascript
- `Number.NaN` Represents Not-a-Number
- `Number.NEGATIVE_INFINITY` Represents negative infinity
- `Number.POSITIVE_INFINITY` Represents infinity

Methods:

- `toExponential(num)` returns a string representing the number in exponential notation
- `toFixed(num)` returns a string representing the number in exponential notation
- `toPrecision(num)` returns a string with the representation of the number with a given precision

Date Object

- Allows you to work with dates and times. An instance must be created in order to work with this object.
- Ways to create a `Date` object
 - `mydate = new Date(milliseconds)`
 - `mydate = new Date()`
 - `mydate = new Date("month day, year hours:minutes:seconds")`
 - `mydate = new Date(yy,mm,dd)`
 - `mydate = new Date(yy,mm,dd,hh,mm,ss)`
- Static methods
 - `Date.now()` Returns the numeric value corresponding to the current number of milliseconds since January 1, 1970, 00:00:00 UTC, ignoring leap seconds.
 - `Date.parse()` Transforms the string representing a date and returns the number of milliseconds since January 1, 1970, 00:00:00 UTC, ignoring leap seconds.

Date Object

- Methods of the Date object
 - `getDate()` returns the day of the month (1-31)
 - `getDay()` returns the day of the week (0 Sunday, 1 Monday,...,6 Saturday)
 - `getHours()` returns the hour (0-23)
 - `getMinutes()` returns the minutes (0-59)
 - `getMonth()` returns the month (0 -11)
 - `getSeconds()` returns the seconds (0-59)
 - `getTime()` returns the milliseconds elapsed since time 0 of 1/1/1970)
 - `getFullYear()` returns the year
 - `toLocaleString()` returns the date as a string in the particular format of the computer
 - `setDate(day)` changes the day of the month.
 - `setHours(hour)` changes the hours
 - `setMinutes(minutes)` changes the minutes.
 - `setMonth(month)` changes the month.
 - `setSeconds(seconds)` changes the seconds.
 - `setTime(milliseconds)` changes the date to milliseconds since 1/1/1970 00:00:00.
 - `setYear(year)` changes the year.

Math Object

- Provides methods for doing mathematical calculations.
- Contains mathematical constants: number e, pi, logarithm to base 10, square root of 2, etc.
- All properties and methods are static
- Math object properties
 - `E` value of number e
 - `LN2` value of the natural logarithm of 2
 - `LN10` value of the natural logarithm of 10
 - `LOG2E` value of the Logarithm in base 2 of E
 - `LOG10E` value of the Logarithm in base 10 of 2
 - `PI` PI number value
 - `SQRT1_2` value of the square root of 0.5
 - `SQRT2` value of the square root of 2
- Example:

`EnumbertimesPInumber = Math.E * Math.PI`

Math Object

- Some methods of the Math object
 - `abs(X)` returns the absolute value of X.
 - `acos(X)` returns the arccosine of X.
 - `asin(X)` returns the arcsine of X.
 - `atan(X)` returns the arctangent of X.
 - `ceil(X)` rounds the number up to the next higher number.
 - `cos(X)` returns the cosine of X.
 - `exp(X)` returns E raised to the number X.
 - `floor(X)` rounds the number to the next lower number.
 - `log(X)` returns the natural logarithm of X.
 - `max(X,Y)` returns the larger of two numbers.
 - `min(X,Y)` returns the smaller of two numbers.
 - `pow(X,Y)` returns X raised to Y.
 - `random()` returns a random number between 0 and 1.
 - `round(X)` rounds the number X to the nearest integer.
 - `sin(X)` returns the sine of X.
 - `sqrt(X)` returns the square root of X.
 - `tan(X)` returns the tangent of X.

String Object

- They are created by means of the constructor `String("literal")`
Ex: `string = new String("my string");`
- Both `String` literals and objects have the `length` property that determines the number of characters in it.
- `String` has the static method `fromCharCode(n1,n2,...)` which creates a string from the characters associated with the Unicode codes n1, n2
- For practical purposes, strings of type "primitive" and objects of type `String` are treated equivalently.
 - Exceptions: `eval(object)` will give the object and `eval(string)` assumes js

String Object

- Some String methods:
 - `charAt(index)` Returns the character at the position specified by `index`.
 - Warning: With **ECMA6** you can access (not delete or assign) the elements of a string as if they were positions in an array: `mystring[2]`
 - `charCodeAt(index)` Returns the encoding number of the character at the position specified by `index`.
 - `indexOf(search_string, index)` Returns the position, within the current string, of the first occurrence of `search_string`, starting at the position indicated by `index`. The `index` argument is optional, if not supplied, the search begins with the first character of the string.
 - `lastIndexOf(lookup_string, index)` Returns the position, within the current string, of the last occurrence of `lookup_string` starting at the position indicated by `index` and looking backwards. The `index` argument is optional, if not supplied, the search begins with the last character in the string.
 - `toLowerCase()` Returns the character string in lowercase.
 - `toUpperCase()` Returns the string in uppercase.
 - `substr(start,length)` Returns the substring starting at the position indicated by `start` and ending at position `start + length - 1`.

String Object

- `substring(firstIndex,secondIndex)` Returns the substring starting at the position given by `firstIndex` and ending at the position before `secondIndex`. If `firstIndex` is less than `secondIndex`, the values are exchanged.
- `slice(firstIndex,secondIndex)` Returns the substring starting at the position given by `firstIndex` and ending at the position before `secondIndex`. If the `secondIndex` is negative, it starts counting from the end of the String
- `concat(string1, string2,...)` concatenates the string with `string1`, `string2`,...
- `split(separator)` Splits the string into an array of character strings. If the separator character or regular expression is not found, it returns an array with a single element that matches the original string.
- `search(ExpReg)` returns the position in the string where the `ExpReg` matches. If there is no match, it returns -1.
- `match(ExpReg)` returns an array with the matches found in the pattern
- `replace(ExpReg/substring,newString)` replaces occurrences of the `ExpReg` or `substring` with `newString`

RegExp Object

- Allows the definition and manipulation of regular expressions in the style of PERL
- To create a regular expression, use the RegExp constructor.

```
expression = new RegExp("\\d+");
expression = new RegExp("[aeiou]+", "gi");
```

- A literal regular expression can be created

```
expression = /[aeiou]+/gi;
```

- RegExp Methods**

- `exec(string)` If there is a match between the pattern and the string, it returns the first match. If invoked again it will return the second match...
- `test(string)` Returns true or false depending on whether or not there is a match between the pattern and the string

- RegExp Properties**

- `globalSpecifies`, `matchmultiline`, `ignoreCaseSpecifies` will contain the value `true` if the corresponding flags have been set in the regular expression
- `lastIndex` It is an index that contains the position of the String where the pattern search will continue
- `source` contains the text of the pattern

ECMA6: Set

- New structure “set” of unique values of any type. If an iterable element is passed as a parameter, its elements are added to the set

```
let myset = new Set();
```

- size** property that returns the number of elements in the set

```
let myset = new Set();
myset.add(8);
myset.add(4);
myset.add(8);
console.log (myset.size); // 2
```

ECMA6: Set

- Methods:
 - `add(newitem)` – Adds the new item to the array
 - `delete (element)` – Removes the element from the set, returning true or false depending on whether it was contained or not.
 - `clear()` – Removes all elements from the set
 - `has(element)` – Returns true or false depending on whether the element belongs to the set or not
 - `entries()` – Returns an iterable containing a set of [key, value] pairs where the keys are equal to the values in insertion order
 - `values()` – Returns an iterable element containing the values of the elements in the order of insertion.
 - `keys()` – Returns an iterable element containing the values of the elements in the order of insertion (same as `values()`).

ECMA6: Map Object

- New iterable structure of key/value pairs
 - `let map = new Map();`
- `size` property that returns the number of elements of the "map"
- Methods:
 - `set(key, value)` – Sets the value of a given key
 - `get(key)` – Retrieves the value of a given key

```
let cart = new Map();
cart.set("Prod1", 5);
cart.set("Prod2", 2);
cart.set("Prod3", 2);
console.log(cart.size);           // 3
console.log (cart.get("Prod2"));  // 2
console.log (cart.get("Prod1"));  // 5
```

ECMA6: Map Object

- Methods:
 - `delete(key)` - Deletes the element associated with the key and returns its value
 - `clear()` - Removes all key/value pairs
 - `has(key)` - Returns true or false depending on whether there is a value associated with the key
 - `entries()` - Returns an iterable containing the set of [key, value] pairs
 - `values()` - Returns an iterable element containing the values of the elements.
 - `keys()` - Returns an iterable element containing the keys of the elements.

```
for (let [key, value] of cart.entries()) {
  console.log("%s has value: %s", key, value);
}
```

Programmer Objects

- An object is an abstraction of the real world characterized by the properties that they identify and the behavior that they have.
- An object is determined by its properties and methods. **Properties** are the characteristics of the object and **methods** are the functions defined to manipulate the properties of the object.
- All objects derive their functionality from the standard **Object**. The mechanism to acquire functionality from other objects is not the classic inheritance of programming languages but is done through **prototyping**.

Object creation (singleton)

- Literal

```
var aPerson = {
  name: "Pepe",
  age: "33",
  gender: "male"
}
```

Property identifiers can be enclosed in quotes

- Dynamic:

```
var aPerson = {};

aPerson.name = "Pepe";
aPerson.age = "33";
aPerson.gender = "male";
```

An empty object is created and properties and methods are added as needed

Objet creation: Constructor (i)

- The properties of the object are defined, with "." notation, by means of a constructor. **this** represents the instance of the object to be created

```
function Person(name, age, gender){
  this.name = name;
  this.age = age;
  this.gender = gender;
}
```

- With operator **new** an object instance is created

```
var aPerson = new Person("Lucía", 27, "woman");
var otherPerson = new Person("Olga", "", "woman");
```

Objet creation: Constructor (ii)

- The properties of the object are defined with associative array notation by means of a constructor

```
function Person(name, age, gender){
    this["name"] = name;
    this["age"] = age;
    this["gender"] = gender;
}
```

- An instance is created using the new operator.

```
var aPerson = new Person("Lucia",27,"woman");
var otherPerson = new Person("Olga", "", "woman");
```

- Indicated to access properties with variables:

```
let apropos = "name"
alert aPerson[apropos] //aPerson.apropos would give an error
```

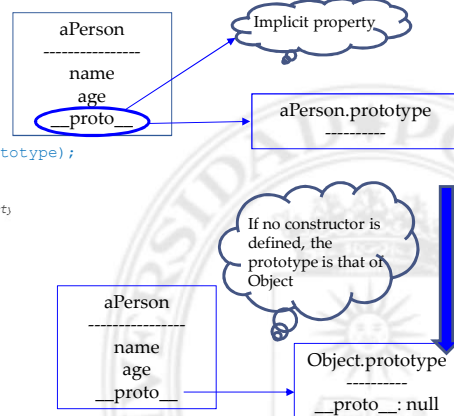
Prototypes System

- Each object has a "prototype" associated with it.
- A prototype is, in turn, an object with information about the hierarchy in which the object is embedded. It is referenced in the `__proto__` property
- The prototypes have, in turn, their own prototype, until reaching null
- The highest level object is Object (default constructor)

```
var aPerson = {
    name:"Pepe",
    age:"33",
}

console.log (aPerson.__proto__);
console.log (aPerson.__proto__ === Object.prototype);
```

```
{constructor: f, __defineGetter__: f, __defineSetter__: f, hasOwnProperty:
f, ...}
  ▶ constructor: f Object()
  ▶ hasOwnProperty: f hasOwnProperty()
  ▶ isPrototypeOf: f isPrototypeOf()
  ▶ propertyIsEnumerable: f propertyIsEnumerable()
  ▶ toLocaleString: f toLocaleString()
  ▶ toString: f toString()
  ▶ valueOf: f valueOf()
  ▶ __defineGetter__: f __defineGetter__()
  ▶ __defineSetter__: f __defineSetter__()
  ▶ __lookupGetter__: f __lookupGetter__()
  ▶ __lookupSetter__: f __lookupSetter__()
  ▶ get __proto__: f __proto__()
  ▶ set __proto__: f __proto__()
```



Prototypes System (ii)

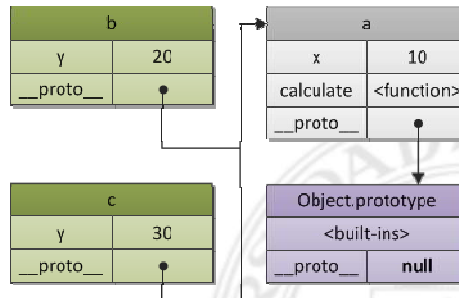
- The prototype can be assigned directly
- <http://dmitysoshnikov.com/ecmascript/javascript-the-core/>

```
var a = {
  x: 10,
  calculate: function (z) {
    return this.x + this.y + z;
  }
};

var b = {
  y: 20,
  __proto__: a
};

var c = {
  y: 30,
  __proto__: a
};

// call the inherited method
b.calculate(30); // 60
c.calculate(40); // 80
```



ATTENTION: If the interpreter does not find a property or method, it requests it from its prototype
this always refers to the object of the scope in which it is executed

Prototypes System (iii)

- Prototypes with constructor:
<http://dmitysoshnikov.com/ecmascript/javascript-the-core/>

```
function Foo(y) {
  // which may create objects
  // by specified pattern: they have after
  // creation own "y" property
  this.y = y;
}

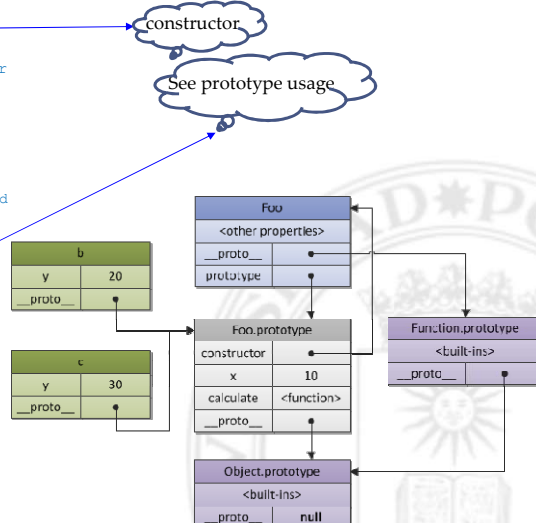
// also "Foo.prototype" stores reference
// to the prototype of newly created objects,
// so we may use it to define shared/inherited
// properties or methods, so the same as in
// previous example we have:

// inherited property "x"
Foo.prototype.x = 10;

// and inherited method "calculate"
Foo.prototype.calculate = function (z) {
  return this.x + this.y + z;
};

var b = new Foo(20);
var c = new Foo(30);

// call the inherited method
b.calculate(30); // 60
c.calculate(40); // 80
```



Object creation: create()

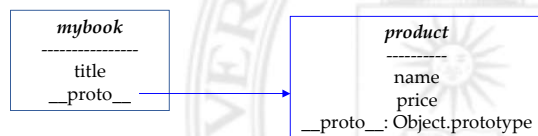
- ES5 standardized the `create()` method of the `Object` object to be able to define new objects:

```
product = Object.create({name:"notebook", price:5});

mybook = Object.create(product, {title: {value:"Cinderella"}});

mybook.name = 'book';
mybook.price = 15;
```

This form can present problems with some types of prototypes (null)



65

Properties and methods

- Access to public properties is done with notation `"."` or associative array notation:


```
Pepe.age = 34;
Pepe["name"] = "Jose";
```
- Methods are defined by assigning to an element of the object the description or identifier of a function in any of the possible formats

```
function getName(){return this.name;}
```

```
function Person(name, age, gender){
  this.name = name;
  this.age = age;
  var category = "friend";
  this.getName = getName;
  this.getAge = function(){return this.age}
}
```

Identifier, **not the invocation**

- Public methods are called with the notation `"."` or associative array notation:


```
var name = Lucia.getName();
var age = Lucia["getAge"]();
```

Private properties and methods

- To define a private variable, declare it as *var* or *let* in the constructor (it is a local variable of the function):

```
function Person(name, age, gender){
  this.name = name;
  this.age = age;
  var category = "friend";
}
```

- Private methods are defined by internal constructor functions that will be called inside.

```
function Person(name, age, gender){
  this.name = name;
  this.age = age;
  this.gender = gender;
  function increment(age)
    {return age + 1}
  this.age = increment(this.age);
}
```

Static methods and properties

- You can have static methods and properties by defining them at the object **constructor**

```
// Static methods and properties

Person.species="Homo Sapiens";

Person.writeSpecies = function() {
  alert('People are of the species' + person.species)
}

Person.writeSpecies();
```

Extending and reducing objects

- An object instance can extend its own properties and methods by defining them

```
var aperson = new Person("Lucia","30","woman");
// New property of the aperson object
aperson.nickname = "Lulu";
// New method of the aperson object
aperson.getNickname = function(){return this.nickname}

alert(aperson.nickname+" "+aperson.getNickname())
```

- An object instance can reduce its properties and methods by using the delete operator on the element to be deleted.

```
var theperson = new Person("Concepcion","70","woman");

// Removing a property from the object
delete(theperson.name);

// Removing a method from the Concepcion object
delete(theperson.getAge);
```

Extending and reducing objects

- ⑩ New properties and methods can be assigned to all objects created with the same constructor by defining them in the constructor's prototype object.

```
var aperson = new Person("Lucia","30","woman");
var anotherperson = new Person("Concepcion","70","woman");

Person.prototype.phone="91";
Person.prototype.setPhone =
    function(phone){this.phone=phone};

aperson.setPhone("888888888888");
alert(aperson.phone+" "+ anotherperson.phone)
```

Prototype use

- If the pattern of object creation through constructor is used, the methods can be created in the prototype of the object so that it is not repeated in each of the instances.
- The **interpreter** calls the prototype when it doesn't find a method or property on an object.

```
function Person(name, age, gender){
  this.name = name;
  this.age = age;
  this.gender = gender;
};
Person.prototype.yearBirth = function(){
  var today = new Date(),
      year;
  year = today.getFullYear();
  return year - this.age;
};
var aperson = new Person("Santiago", 35, "H");
console.log("The year of birth is: " + aperson.yearBirth());
```

The prototype is an object and therefore properties and methods can be added to it.

Inheritance from an object

- For a child object to inherit the properties and methods of another **parent** object, it is necessary to create a parent object and **assign it to the prototype object** of the child object's constructor.

```
child.prototype = new parent();
// this way the created hierarchy is respected
// (if it has methods on the prototype)
```

- In order to invoke the constructor of the parent object in the child object, it is necessary to use **apply() or call()** that allow executing a function (in this case the constructor of the parent) with the **context** that we pass as the first parameter and the **arguments** of the second (is an array)
method.apply(context, array arguments) calls the method with the arguments in the defined context
method.call(context, arg1, ..., argN) similar to apply but the arguments are passed individually instead of in an array

- Ex.:

```
Person.apply(this, [name, age, sex]);
```

Applies the Person constructor on the current object and passing the name, age, and gender as parameters

Inheritance using prototypes

```
function Polygon(height, width){                                // father
    this.height = height;
    this.width = width;
}

function Square(side){                                         // child
    this.prototype = new Polygon();
    this.getArea = function(){
        return this.height * this.width;
    }
    Polygon.call(this, side, side);
}

let mysquare = new Square(20);
console.log (mysquare.getArea()); //400
console.log (mysquare.width);     //20
```

ECMA6: Object creation

- Objects can be defined simply if the value matches the property name

```
let myObj = {prop1, prop2}
```

Is equivalent to:

```
let myObj = {prop1: prop1, prop2:prop2}
```

- Property names can be created dynamically :

```
function getanum(){
    return parseInt(Math.random()*10);
}
```

```
let myObj = {
    prop1: "valueProp1",
    ["prop" + getanum()]: "valueProp2"
}
```

```
for (prop in myObj){
    if (myObj.hasOwnProperty(prop))
        console.log(prop + ": " + myObj[prop]);
}
```

ECMA6: Object creation (ii)

- Object methods can be defined directly

```
let aperson = {
  name: "Santiago",
  birthyear: 1970,
  age() {
    let today = new Date(),
        currentyear = today.getFullYear(),
        theage = currentyear - this.birthyear;
    return theage;
  }
}
console.log (aperson.age()) ;
```

ECMA6: Classes

- ECMAScript 6 allows the definition of classes with an intuitive syntax
- They are defined by the **class** keyword and the class body is enclosed in curly braces {}
- Within the body, the methods and constructors are defined.
- The constructor is defined by the **constructor** keyword and there can only be one

```
class Product {
  constructor (name, price, stock) {
    this.name = name;
    this.price = price;
    this.stock = stock;
  }
  setSale(quantity) {
    this.stock = this.stock - quantity;
    return this.price * quantity;
  }
}

const aProducto = new Product ('memory', 100, 25),
      soldOut = 2;
console.info ("Sale price: " + aProducto.setSale(soldOut));
console.info ("There are left: " + aProducto.stock);
```

ECMA6: Classes (ii)

miw.etsisi.upm.es • 77

- Static methods can be defined using the static keyword.

```
class Product {
  constructor (name, price, stock) {
    this.name = name;
    this.price = price;
    this.stock = stock;
  }
  setSale(quantity) {
    this.stock = this.stock - quantity;
    return this.price * quantity;
  }
  static getVAT(product, quantity) {
    const vat = 0.21;
    const unitaryVAT = product.price * vat;
    return unitaryVAT * quantity;
  }
}
```

```
const aProduct = new Product ('memory', 100, 25),
      soldOut = 2;
console.info ("Sale price: " + aProduct.setSale(soldOut));
console.info ("VAT: " + aProduct.getVAT(aProduct, soldOut));
console.info ("There are left: " + aProduct.stock);
```

ECMA6: Classes (iii)

miw.etsisi.upm.es • 78

- Subclasses are defined using the **extends** keyword, and within a subclass the superclass can be referenced via the **super** keyword.

```
class Product {
  constructor (name, price, stock) {
    this.name = name;
    this.price = price;
    this.stock = stock;
  }
  getVAT() {
    const VAT = 0.21;
    var myvat = this.price * VAT;
    return myvat;
  }
  setStock(quantity) {
    this.stock = this.stock - quantity;
  }
  setSale(quantity) {
    var totalCant = 0;
    this.setStock(quantity);
    totalCant = (this.price +
      this.getVAT()) * quantity;
    return totalCant;
  }
}

class Book extends Product {
  constructor (name, price, stock, title) {
    super(name, price, stock);
    this.title = title;
  }
  getVAT() {
    const VAT = 0.04;
    var myvat = this.price * VAT;
    return myvat;
  }
}
```

ECMA6: Classes (iv)

```
class Employee {
  constructor (name, salary){
    this.name = name;
    this.salary = salary;
  }
  getTaxes(){
    return this.salary * 0.15;
  }
  static salaryDifference (Emp1, Emp2){
    var difference = Math.abs(Emp1.salary - Emp2.salary);
    return difference;
  }
}
```

```
class Engineer extends Employee {
  constructor(name, salary, category){
    super (name, salary);
    this.category = category;
  }
  getTaxes() {
    return super.getTaxes() + this.salary * 0.05;
  }
}
```

ECMA6: Classes (v)

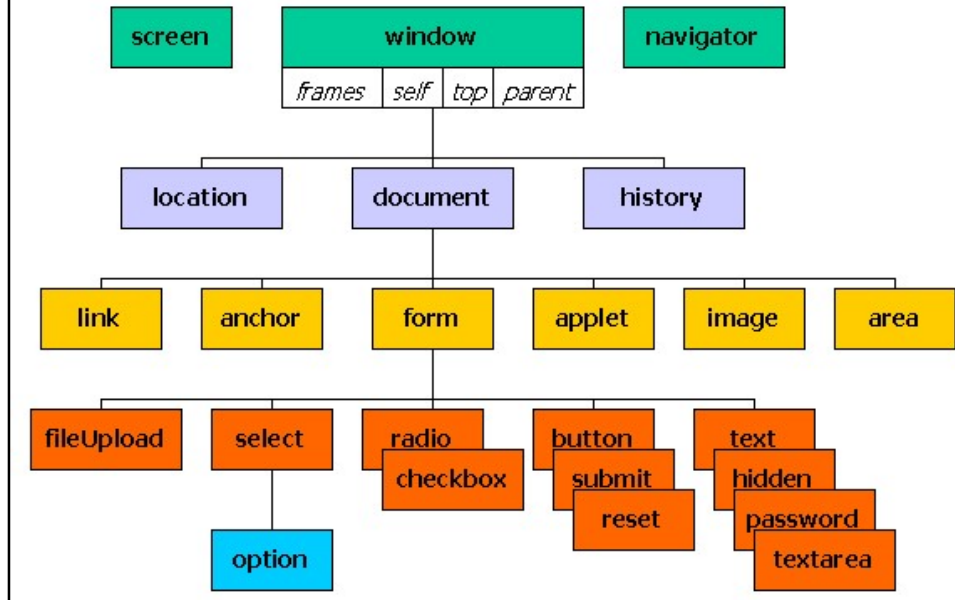
- The **setter** and **getter** methods can be defined directly:

```
class Enemy {
  constructor(name,power){
    this._name = name;
    this._power = power;
  }
  get name(){
    return this._name;
  }
  set name(name){
    this._name = name;
  }
  get power(){
    return this._power;
  }
  set power(power){
    this._power = power;
  }
}
```

```
var anEnemy = new Enemy('theEnemy', 100);
console.log ("Enemy is named: " + anEnemy.name + " and has power: " + anEnemy.power);
```

ATTENTION:
property
names are not
what is used
after

BOM(Browser Object Model)



Accessing a page element (i)

- To access an element you have to obtain a reference to it and then access its properties or methods:
 - `document.getElementById("identifier")` returns a reference to the element corresponding to the HTML element that has the id attribute equal to the value "identifier"
- You can also get arrays of references to elements that have something in common (name in the case of a "radio button" or that have the same HTML tag:
 - `document.getElementsByName("name")`, returns an array of references to the corresponding elements with the name attribute equal to the value "name"
 - `document.getElementsByTagName("tag")`, returns an array of references to the elements corresponding to the HTML "tag"

Html

```

```

javascript

```
document.getElementById("myimage").src = "another.gif";
```

Accessing a page element (ii)

Accessing a text type element :

- Html

```
<form name="myform">
  <input type="text" name="mybox" id="idbox">
</form>
```

Value of the element

- Javascript

```
document.getElementById("idbox").value = "Hello Santiago";
alert(document.getElementById("idbox").value);
```

Or

```
var my_input = document.getElementById("idbox");
my_input.value = "Hello Santiago";
alert (my_input.value);
```

Reference to the element

BOM: screen object

- The **screen** object has the following properties:
 - **width** contains the width in pixels of the screen.
 - **height** contains the height in pixels of the screen.
 - **left** contains the left position where the window can be placed.
 - **top** contains the top position where the window can be placed.
 - **colorDepth** Contains the number of bits of the palette being used on the screen.

BOM: navigator object

- Properties of the navigator object:
 - `appName`, name of the base code of the browser
 - `appVersion`, browser name
 - `appVersion`, browser version
 - `userAgent`, information sent by the browser to the server when a web page is requested
 - `mimeTypes[i].type`, i-th element of the array of mime types recognized by the browser
 - `plugins[i].name`, i-th element of the array of plugins recognized by the browser
 - `userLanguage`, browser language version
 - `platform`, operating system on which it runs
 - `online`, boolean indicating if the browser is online
 - `cookiesEnabled`, boolean indicating if cookies are enabled

BOM: window object

- window object properties
 - `closed`, boolean indicating if the window is closed.
 - `defaultStatus`, default message in the status bar.
 - `frames`, array representing the frame objects of the window object.
 - `history`, array representing the URLs stored in the history of the window object.
 - `length`, number of frames
 - `name`, name of the current window or frame.
 - `opener`, a reference to the window object that opened the current window object (if the `open()` method was used to do so).
 - `parent`, reference to the window object that contains frameset.
 - `self`, Alternative name of the current window object.
 - `status`, status bar message in an instant.
 - `top`, Alternative name for the topmost window.
 - `window`, Alternative name of the current window object.

BOM: window object

- window object methods
 - `alert(message)`, displays the message in a dialog window.
 - `confirm(message)`, displays the message in a dialog window with options to "OK" and "Cancel".
 - `prompt(message, response)`, displays the message in a dialog window and captures the user's response.
 - `blur()` removes the focus from the window object.
 - `focus()`, captures the focus on the window object.
 - `setInterval("expression", time)`, Evaluates the expression every time the specified milliseconds pass in time. returns an id
 - `setTimeout("expression", time)`, Evaluates the expression after the specified milliseconds have passed in time. Returns an ID.
 - `clearInterval(ID)`, Removes the timer referenced by ID.
 - `clearTimeout(ID)`, Cancels the timer specified by ID.

BOM: objeto window

- ◎ Métodos del objeto `window`
 - `moveBy(x,y)`, moves the current window object the number of pixels specified by x and y.
 - `moveTo(x,y)`, moves the current window object to the coordinates specified by x and y.
 - `resizeBy(x,y)` resizes the current window object by moving its lower right corner by the number of pixels specified by x and y.
 - `resizeTo(width,height)`, resizes the current window object by changing the `outerWidth` and `outerHeight` values to the specified width and height values.
 - `scroll(x,y)`, scrolls the current window object to the coordinates specified by x and y.
 - `scrollBy(x,y)`, scrolls the current window object by the number of pixels specified by x and y.
 - `scrollTo(x,y)`, scrolls the current window object to the coordinates specified by x and y.
 - `close()` Closes the current window object.
 - `print()`, opens the dialog window for printing the document

BOM: objeto window

- **open(url, name, characteristics)**, method that loads the url in the window named name with the specified characteristics. Optional features are:
 - **toolbar = [yes,no,1,0]** - The window has a toolbar.
 - **location = [yes,no,1,0]**: the window has a location field.
 - **directories = [yes,no,1,0]**: The window has direction buttons.
 - **status = [yes,no,1,0]**: the window has a status bar.
 - **menubar = [yes,no,1,0]**: the window has a menu bar.
 - **scrollbars = [yes,no,1,0]** - The window has scrollbars.
 - **resizable = [yes,no,1,0]** - The window will be resizable.
 - **width = pixels**: Indicates the width of the client window in pixels.
 - **height = pixels**: Indicates the height of the client window in pixels.
 - **outerWidth = pixels**: Indicates the total width of the window in pixels.
 - **outerHeight = pixels**: Indicates the total height of the window in pixels.
 - **left = pixels**: distance in pixels from the left side of the screen that the window should be placed.
 - **top = pixels**: distance in pixels from the top of the screen at which the window should be placed.

BOM: Objeto location

- ⦿ Properties of the location object:
 - **hash**, name of the internal link (#link) within the URL.
 - **host**, server name, and port number from the URL.
 - **hostname**, Is a string that contains the domain name of the server (or IP address), within the URL.
 - **href**, is a string containing the full URL.
 - **pathname**, The path to the resource, within the URL.
 - **port**, the port number of the server, within the URL.
 - **protocol**, the protocol used within the URL.
 - **search**, query-string within the URL.
- ⦿ Methods of the location object:
 - **reload()** reloads the URL specified in the href property of the location object.
 - **replace(urlString)**, replaces the current history while loading the URL specified in urlString.
- ⦿ Changing the location value of a window or frame loads another page in it.

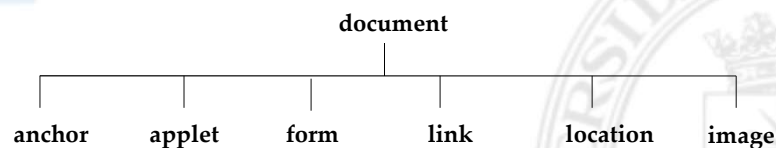

```
window.location="url"
```

BOM: history object

- The history object stores the list of sites that have been "browsed".
- This object is used to manage that list, being able to carry out movements forwards and backwards in it.
- history object methods:
 - `back()`, reloads the URL of the previous document within the history.
 - `forward()`, reloads the URL of the next document within the history.
 - `go(position)`, reloads the URL of the specified document by position within the history.

BOM: document object

- The `document` object represents any document loaded in a window or frame.
- `document` object hierarchy:



BOM: objeto document

- **document** object properties
 - **alinkColor**, Represents the color of the active links.
 - **anchors**, array containing internal links of the document.
 - **applets**, array containing all the applets of the document.
 - **bgColor**, Represents the background color of the document.
 - **cookie**, string with the values of the cookies of the current document.
 - **domain**, is the name of the server that has served the document.
 - **embeds**, array containing the <EMBED> elements of the document.
 - **fgColor**, Represents the foreground color of the document.
 - **forms**, array containing all form objects of the document.
 - **Images**, array containing all the images of the document.
 - **lastModified**, date of the last modification of the document.
 - **linkColor**, represents the color of the links in the document.
 - **links**, array containing the external links of the document.
 - **location**: Is a string containing the URL of the current document.
 - **referrer**, string containing the URL of the document that called the current one, if the user used a link.
 - **title**, string containing the title of the current document.
 - **vlinkColor**, represents the color of the visited links.

BOM: document object

- **document** methods
 - **clear()**, clears the document window.
 - **close()**, closes the writing on the current document.
 - **open(mime,"replace")**, opens writing on a document. The mime parameter specifies the document type supported by the browser. If the string "replace" is passed as the second parameter, the previous document within the history is reused.
 - **write()** Writes text and HTML to the current document.
 - **writeln()** Writes text and HTML to the current document, followed by a newline.
 - **getElementById("identifier")** accesses the HTML element that has the id attribute with the value of "identifier"
 - **getElementsByName("name")** Accesses all HTML elements that have the value "name" in the name attribute
 - **getElementsByTagName("tag")** Accesses all HTML elements of type specified by "tag"

BOM: form object

- ◉ The **form** object contains all the information about a form in a document.
- ◉ properties of the **form** object
 - ◉ **action**, string specifying the URL where the form information should be processed/submitted.
 - ◉ **elements**: array that contains each of the objects that appear in the form, and in the same order in which they appear in the HTML code.
 - ◉ **encoding**, string containing the MIME encoding specified in the ENCTYPE attribute.
 - ◉ **method**, string containing the name of the method in charge of receiving/processing the form information.
 - ◉ **target**, string containing the name of the window to which the form responses should be sent.
- ◉ Methods of the **form** object
 - ◉ **reset()**, resets the form. It produces the same result as clicking on a RESET type button.
 - ◉ **submit()**, submits the form. It produces the same result as clicking on a button of type SUBMIT.

BOM: text, password and textarea objects

- ◉ Properties of the **text**, **password** and **textarea** objects
 - ◉ **name**, string containing the name of the text, password or textarea object, that is, it contains the value of the NAME attribute.
 - ◉ **defaultValue**, string containing the default value of the object
 - ◉ **value**, string that contains the value of the text, password or textarea object, at each moment. This property is also held by the hidden and file objects.
 - ◉ **disabled**, boolean indicating whether the element is disabled
 - ◉ **readonly**, boolean indicating if the element is readonly
 - ◉ **type**, control type
 - ◉ **form**, reference to the form where the object is included (they also have hidden and file)
- ◉ Methods of the text, password and textarea objects
 - ◉ **blur()**, fires the event of removing the focus of the given object
 - ◉ **click()**, fires the event of removing the focus of the given object.
 - ◉ **focus()**, fires the event of getting the focus on the given object.
 - ◉ **select()**, fires the event of selecting the object's text.
- ◉ **textarea** has properties to access cols and rows
- ◉ **text** and **password** have properties to access **size** and **maxlength**

BOM: submit, reset and button objects

• Properties of the **submit**, **reset** and **button** objects

- **name**, string containing the name of the object, that is, the value of the NAME attribute.
- **value**, string containing the value of the button object, that is, the value of the VALUE attribute.
- **type**, control type
- **form**, reference to the form where the object is included

• Methods of the **submit**, **reset** and **button** objects

- **click()**, fires the event of clicking on the button



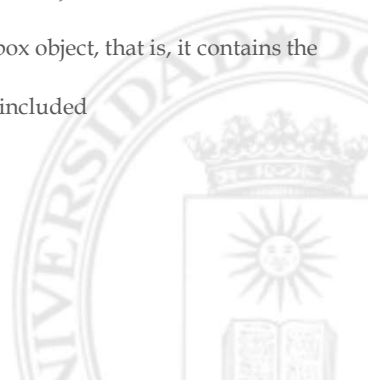
BOM: checkbox object

• **checkbox** object properties

- **checked**, boolean value that indicates if the checkbox object is selected or not.
- **defaultChecked**, boolean value that indicates if the checkbox object should be selected by default.
- **name**, string containing the name of the checkbox object, that is, it contains the value of the NAME attribute.
- **value**, string containing the value of the checkbox object, that is, it contains the value of the VALUE attribute.
- **form**, reference to the form where the object is included

• **checkbox** object methods

- **click()**, fires the click event on the object



BOM: radio object

- ◉ Groups the radio elements with the **same value of the name** attribute.
- ◉ It is really an **array** in which each item corresponds to one of the grouped radio elements
- ◉ **Radio** object properties
 - ◉ **length**, numerical value that indicates the number of options.
 - ◉ **form**, reference to the form where the object is included
 - ◉ **radioname[i]** contains:
 - ◉ **checked**, boolean value that indicates if the radio object is selected or not.
 - ◉ **defaultChecked**, boolean that indicates if it should be selected by default.
 - ◉ **name**, string containing the name of the radio object (NAME attribute).
 - ◉ **value**, string containing the value of the radio object (VALUE attribute).
- ◉ Radio Object Methods
 - ◉ **click()**, fires the event of clicking on an action that must be applied to an item of the radio(**radioname[i].click()**).

BOM: select object

- **select** object properties
 - **length**, is a numeric value that indicates the number of options of the given select object.
 - **form**, reference to the form where the object is included
 - **multiple**, boolean that indicates if the select is multiple
 - **size**, contains containing the value of the SIZE attribute
 - **name**, is a string that contains the name of the select object, that is, it contains the value of the NAME attribute.
 - **selectedIndex**, It is a numerical value that indicates the index of the currently selected option.
 - **options**, is an array with the select options

BOM: options array from select object

- **options**: array containing each of the options that appear in the selectable list. It has the following properties:
 - **defaultSelected**, indicates if the option was selected by default.
 - **index**, numeric value indicating the index of the option within the list.
 - **length**, numeric value indicating the number of options in the selection.
 - **options**, string containing the HTML code for the selection list.
 - **selected**, boolean value indicating whether the option is selected or not.
 - **selectedIndex**, numeric value: indicates the index to the selected option.
 - **text**, string containing the text displayed in the list for each option.
 - **value**, contains the value (**VALUE** attribute) of a particular option

Image Object

- Allows you to store images.
- It is used to perform image preloads.
- By means of constructor **Image(A,B)** an image object of size **A x B** is created without an associated image file. **size** is optional
- Using the **src** property an image file is assigned to the object

```
myimagen = new Image(100,100);
myimagen.src= "image.gif";
```

Events

- Events are actions or occurrences that happen on the system you are programming and that the system informs you of so that you can respond in some way if you wish.
 - https://developer.mozilla.org/en/docs/Learn/JavaScript/Building_blocks/Events
- An event is an event that occurs in a browser when the user does something internal to the browser itself (loading or downloading a Web page)
- Events can be fired by the user (press a key, move a mouse, etc) or by javascript code:


```
form.submit()
```
- In javascript, each event has an associated event handler that, associated with code, causes such code to be executed if its associated event occurs.

Events

- Some event handlers:

- onabort	- onmousedown
- onblur	- onmousemove
- onchange	- onmouseup
- onclick	- onmouseover
- ondblclick	- onmouseout
- onerror	- onreset
- onfocus	- onresize
- onkeydown	- onselect
- onkeypress	- onsubmit
- onkeyup	- onunload
- onload	

There are specific event handlers:

onmessage (*webworkers*); ondragstart, ondragover, ondrop, ondragenter, ondragleave (*drag and drop*); loadstart, loadend, progress (*file api*), storage (*webstorage*), etc.

Events and event handlers

- **onclick** occurs when the user clicks on an HTML element.
 - **onSubmit** is fired when the user clicks the button of type submit.
 - **onReset** is activated when the reset button has been pressed.
 - **onBlur** fires when an object is no longer the active element of a form.
 - **onFocus** is the opposite of a blur event, it fires when the focus is on an element.
 - **onChange** fires when the user changes something in a form element.
 - **onSelect** fires when a text is selected
 - **onMouseOver** fires when the mouse is hovered over any link or area of an object.
 - **onMouseOut** fires when the mouse is positioned outside of a link or area of an object.
 - **onMouseDown** fires when the mouse button is pressed.
 - **onMouseUp** fires when the mouse button is released.
 - **onMouseMove** fires when the mouse is moved.
 - **onKeyDown** fires when a keyboard key is pressed.
 - **onKeyUp** fires when a pressed keyboard key is released.
 - **onKeyPress** fires when the user holds down a key.
 - **onLoad** handler is executed when the containing page or image is finished loading.
 - **onUnload** handles the event that occurs when a window or frame is exited.
- Complete list in: <https://developer.mozilla.org/es/docs/Web/Events>
 - <https://sites.google.com/site/dwebtodojs/referencia/modelo-de-eventos-del-dom>

Event handlers

- In the html tag of the element on which you want to detect the event:

```
<TAG atributos onevento = "Event processing">
Ej.:
function warn(){ alert ('You clicked')}
<input type='text' id='myfield' onkeypress = "warn()">
```

function call

- As an element property:

```
<input type='text' id='myfield' />
document.getElementById('myfield').onkeypress = warn,
```

function name

- Defining the event handler on the element

```
addEventListener(eventtype, function, useCapture)
  ▫ eventtype event type (without "on")
  ▫ funcion this is the name of the event handler
  ▫ useCapture: true or false, phase in which the event will be processed (capturing or bubbling)
```

```
document.getElementById('myfield').addEventListener(keypress, warn, false)
```

function name

In the code that processes an event, the **this** reference is to the element that captures the event

Event handlers as properties

- Event handlers can be assigned in javascript as if they were properties:

```
document.getElementById("elem").onclick = theFunction;
```

Attention: the name of the function to be executed does not include parentheses since if it had them, the result of its execution would be assigned to the event

- To run something on load you can use:

```
window.onload = theFunction;
```

And it is equivalent to: `<body onload = "theFunction()">`

// or an anonymous function:

```
window.onload = function(){  
  // code to execute  
}
```



Event handlers setting up

- Setting up event handlers: The `addEventListener()` method of the object on which the event is to be treated is used

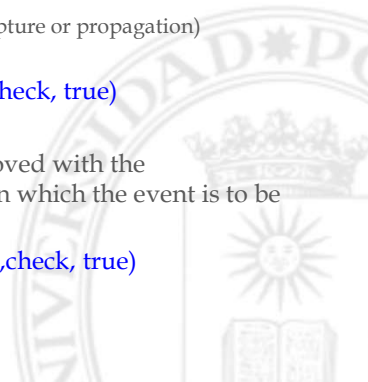
```
addEventListener(eventType, myfunction, useCapture)
```

- `eventType` is the event type without "on"
- `myfunction` is the "handler" of the event
- `useCapture`: true or false, phase in which it is (capture or propagation)

Ex: `document.addEventListener("click",check, true)`

- Removal of event handlers : A handler is removed with the `removeEventListener()` method of the object on which the event is to be treated

```
document.removeEventListener("click",check, true)
```



EVENT OBJECT: Advanced management

- The interpreter maintains an object called *event* with properties about the event that occurred that are updated each time the event occurs.
- According to the standard, this object is automatically passed as a parameter to the function that must be executed to process the event that has occurred.

```
document.getElementById("elem").onclick = TreatEvent;
```

```
function TreatEvent (eventObject) {
    // Here the event would be treated
    // and eventObject is the event object
    ....
}
```

- **Warning:** in older versions of Internet Explorer the object exists as a global object, called *window.event*, and is not passed as a parameter to the function. In order to solve it:

```
function TreatEvent (TheEvent){
    var event = TheEvent || window.event;
    // Here the event would be processed
}
document.onclick = TreatEvent; // fires when clicking on the document
```

Event object: properties

- **type:** type of event occurred (same as the name of the event but without "on", for example mousedown, click, etc)
- **target:** is a reference to the element on which the event that has been fired has occurred. You have access to its properties (for example id of the element on which it has been triggered)
- **currentTarget:** is a reference to the element that handles the event (it doesn't have to be the one that fires it)
- **button:** determines the button pressed on the mouse (0 – none, 1-left, 2-right, 3- left and right, 4- center, etc)
- **clientX, clientY:** coordinates in which the event occurs with respect to the browser window. Does not take into account the scroll made
- **screenX, screenY:** coordinates in which the event occurs with respect to the entire screen
- **altKey:** boolean value that determines if the Alt key was pressed at the time the event occurred
- **ctrlKey:** boolean value that determines if the Control key was pressed at the time the event occurred
- **shiftKey:** boolean value that determines if the Shift key was pressed at the time the event occurred
- **eventPhase:** current propagation phase (catch or bubble)
- **timestamp:** Date type object that determines when the event has occurred.
- **detail:** number of mouse clicks 1 for a click, 2 for a double click,...

(The drag events have the *dataTransfer* property in which information is stored)

Event object: properties

The detection of the pressed key presents some complication

- **charCode**: returns the Unicode code of the character of the pressed key if it corresponds to a character. Not defined for keyup and keydown.
- **keyCode**: returns the numeric code of the pressed key in uppercase. Works with non-character keys.
- **key**: returns the character of the pressed letter if it corresponds to a character. Works with non-character keys (ArrowRight, ArrowLeft, ...).
- **which**: returns the **charCode** or the **keyCode** depending on whether the key is alphanumeric or not

It is common to match the treatment by doing:

```
function showKey (theEvent){
  var event = theEvent || window.event;
  var code = event.charCode || event.keyCode;
  character = String.fromCharCode(code);
  alert (character)
}
```

```
document.onkeypress = showKey;
```

During *keypress* the press of non-alphanumeric keys is not detected

In modern browsers:

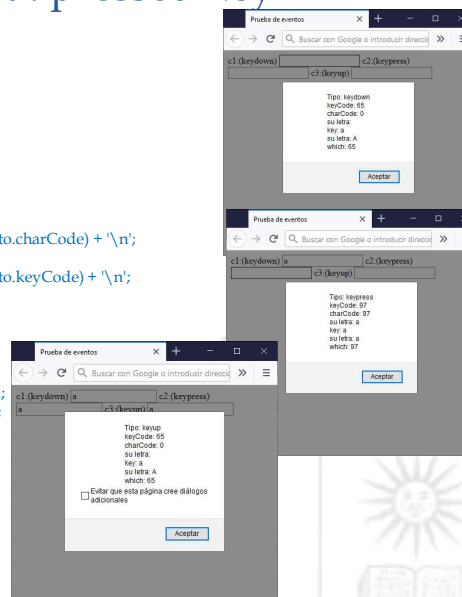
```
function showKey (theEvent){
  var character= theEvent.key;
  alert (character)
}
```

```
document.onkeydown = showKey;
```

event object : pressed key

```
<html>
<head>
  <title>Prueba de eventos</title>
  <meta charset='utf-8'/>
</head>
<body>
<script type='application/javascript'>
function Procesa(ElEvento){
  let mensaje = 'Tipo: ' + ElEvento.type + '\n';
  mensaje += 'keyCode: ' + ElEvento.keyCode + '\n';
  mensaje += 'charCode: ' + ElEvento.charCode + '\n';
  mensaje += 'su letra: ' + String.fromCharCode(ElEvento.charCode) + '\n';
  mensaje += 'key: ' + ElEvento.key + '\n';
  mensaje += 'su letra: ' + String.fromCharCode(ElEvento.keyCode) + '\n';
  mensaje += 'which: ' + ElEvento.which + '\n';
  alert (mensaje);
} // de la función

window.onload = function(){
  document.getElementById('c1').onkeydown = Procesa;
  document.getElementById('c2').onkeypress = Procesa;
  document.getElementById('c3').onkeyup = Procesa;
}
</script>
<form>
  c1:(keydown) <input type='text' id='c1' />
  c2:(keypress) <input type='text' id='c2' />
  c3:(keyup) <input type='text' id='c3' />
</form>
</body>
</html>
```



Event cancellation

- It can be canceled if the return of the javascript is a **false** value
 - If you want to prevent a form from being submitted:

<Form name="myform" onsubmit="return Check()">

"Check" should return **true** if everything went well and **false** if the form should not be submitted

In general, if the event handler returns false, the event that the event handler was fired by will be aborted.

- The `preventDefault()` method of the event object is executed in the code:
 - `event.preventDefault()`
 - `if (!valid) event.preventDefault()`

Event cancellation example: return false

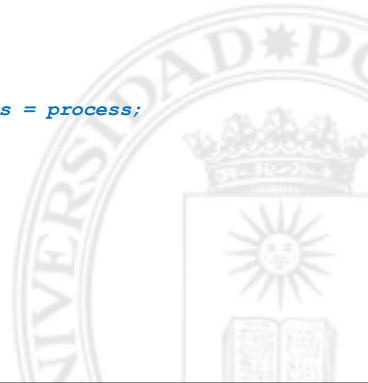
```
<html><head><title>Event test</title>
</head>
<body>
<script type='application/javascript'>
  function process(theEvent){
    var mykey = theEvent.key
    if (mykey < '0' || mykey > '9'){
      alert ('I told you just numbers!!!');
      return false;
    } // if mykey is out of range
    return true;
  } // function

window.onload = function(){
  document.getElementById('myfield').onkeypress = process;
}
</script>
<form>
  Just numbers field
  <input type='text' id='myfield' />
</form>
</body>
</html>
```

Event cancellation example : *preventDefault()*

```
<html><head><title>Event test</title>
</head>
<body>
<script type='application/javascript'>
  function process(theEvent){
    var mykey = theEvent.key
    if (mykey < '0' || mykey > '9'){
      alert ('I told you just numbers');
      theEvent.preventDefault();
    }
  }

  window.onload = function(){
    document.getElementById('myfield').onkeypress = process;
  }
</script>
<form>
  Just numbers fields:
  <input type='text' id='myfield' />
</form>
</body>
</html>
```



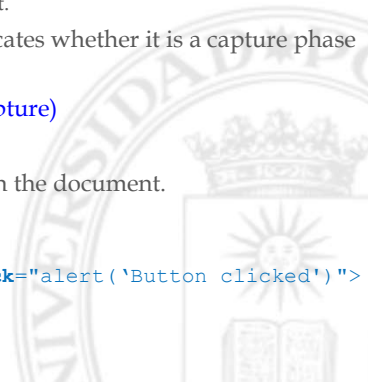
Event propagation

- Event propagation is done in two phases:
 - The first phase is the capture phase where the event propagates through the document tree following its descendants until it reaches the target node. During propagation, the defined event handlers are executed. In some versions of Internet Explorer there was no capture phase
 - In the second phase, bubbling, the event propagates from the innermost element that has dealt with it until it reaches the root of the document.
- In the handler definition, the third parameter indicates whether it is a capture phase (true) or a bubbling phase (false):

`addEventListener(eventType, myfunction, useCapture)`
- Example:

If you click on a button on a form, you also click on the document.

```
<body onclick="alert(Document clicked)" ">
  <form>
    <Input type='Button' Value='Click' onclick="alert('Button clicked') ">
  </form>
```



Event bubbles: propagation control

- Propagation can be canceled with the `stopPropagation()` method of the *event* object.

```
<body>
<form>
  <input id='myb' type='button' value='Click here'>
</form>
<script>
```

```
function buttonEvent(event){
  alert ("Button clicked");
  event.stopPropagation();
}
```

This function will never be executed as it was stopped the propagation

```
function documentEvent(event){
  alert ("Document clicked");
  alert ("At coordinates: "+event.clientX + ","+event.clientY);
}
```

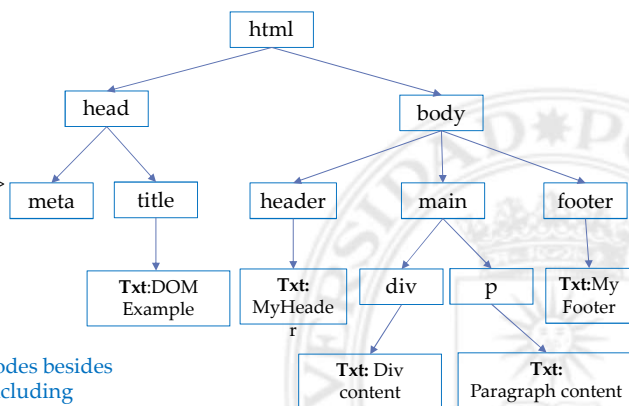
```
window.onload = function(){
  document.getElementById("myb").addEventListener("click", buttonEvent, false);
  document.addEventListener("click", documentEvent, false);
}
```

`event.cancelBubble` is equivalent for old browsers (IE from Microsoft)

Document Object Model (DOM)

- When an HTML document is finished loading, JavaScript creates a tree that represents the document hierarchy with all the HTML elements
- In general, for each label with text, a node corresponding to the label is created from which another node with the text hangs.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset='utf-8' />
    <title> DOM Example</title>
  </head>
  <body>
    <header>MyHeader</header>
    <main>
      <div>Div content</div>
      <p>Paragraph content</p>
    </main>
    <footer>MyFooter</footer>
  </body>
</html>
```



There are more types of nodes besides element and text nodes, including attribute nodes.

DOM2: accessing HTML elements

- `document.getElementById("identifier")` returns a reference to the node corresponding to the HTML element with the **id attribute** equal to the value **"identifier"**
- `document.getElementsByName("name")`, returns an array of references to the corresponding nodes with HTML elements with the **name attribute** equal to the value **"name"**
- `document.getElementsByTagName("tag")`, returns an array of references to nodes corresponding to the HTML **"tag"**
- `refnode.getElementsByTagName("class")`, returns an array of references to nodes with HTML elements with the **class attribute** equal to the value **"class"** that are descendants of **refnode**.
- `refnode.querySelectorAll("selectorCSS")`, returns an array of references to nodes that match the CSS selector provided as a parameter that are descendants of **refnode**.

DOM2: nodes properties

- **nodeName**, The name of the node, such as HEAD for the head element.
- **nodeValue**, If not a node corresponding to an HTML element (`nodeType = 1`), contains the value of the node. Ex: if it is a text node (`nodeType = 3`) it contains the associated text
- **nodeType**, The numeric type of the node. The most important ones are: 1 = `ELEMENT_NODE`, 2 = `ATTRIBUTE_NODE`, 3 = `TEXT_NODE`, 8 = `COMMENT_NODE`, 9 = `DOCUMENT_NODE` and 10 = `DOCUMENT_TYPE_NODE`
- **parentNode**, Reference to the parent node of the current node.
- **childNodes**, List of child nodes of the parent node
- **firstChild**, Reference to the first child node of the current one.
- **lastChild**, Reference to the last child node of the current one.
- **previousSibling**, Reference to the previous sibling in a list of nodes.
- **nextSibling**, Reference to the next sibling in a list of nodes.
- **attributes**, A list of key/value pairs with the attributes of the corresponding nodes with HTML elements

DOM2: node management

- `document.createElement(tag)` creates a **node of type HTML** element corresponding to the tag and returns a **reference** to it
- `document.createTextNode(content)` creates a **text node** with the specified content and returns a **reference** to it
- `parentNode.insertBefore(newNode,reference)`, inserts **newNode** before the node specified by reference in the applied context (**parentNode**) and returns a **reference** to it
- `parentNode.replaceChild(newNode,oldNode)`, replaces **oldNode** with **newNode** in the applied context (**parentNode**). Returns a reference to **oldNode**
- `parentNode.removeChild(oldNode)`, removes **oldNode** in the applied context (**parentNode**). Returns a reference to **oldNode**
- `parentNode.appendChild(newNode)`, adds a new descendant by the end inside **parentNode**

DOM2: node management (ii)

- `element.outerHTML(string)` replaces the element in the DOM tree with the result of "parsing" the string
- `element.insertAdjacentHTML(place, string)` allows to insert the HTML string in the indicated place with respect to the element. Place values can be:
 - `beforeBegin` : insert before the element
 - `afterBegin` : insert as the first child of the element
 - `beforeEnd` : insert as the last child of the element
 - `afterEnd` : insert after the element
- `element.insertAdjacentElement(place, anotherelement)` allows to insert **anotherelement** (via its reference) in the corresponding place of **element**

DOM2: node attributes

- `getAttribute(name)` retrieves the **value** of the attribute **name**
- `setAttribute(name,value)` assigns the value to the attribute **name**
- `removeAttribute(name)` removes the attribute **name**
- `hasAttribute(name)` returns a boolean indicating whether the element has the attribute **name**

Content rewriting

- The content of an element can **be known and modified** with the **innerHTML** property.
- The **innerHTML** property includes the HTML tags.
- It is very frequently used in **AJAX**

```
<div id= "mylayer" style='border:solid 1px black;color:red'>
  This is the content<br>
  <b> of the layer I am watching </b>
</div>
```

```
alert (document.getElementById("mylayer").innerHTML;
```

```
var astring= "This is the <i>new content</i> for the layer";
```

```
document.getElementById("mylayer").innerHTML = astring;
```

Modifying a CSS property value

- You can modify the value of any CSS property of an element through the element's **style** object, which contains all the CSS properties:

```
document.getElementById('unaid').style.color = 'red'
```

- Since it is treated as an object, it can be accessed with associative **array** notation:

```
document.getElementById('unaid').style['color'] = 'red'
```

- ATTENTION: If you want to manipulate a CSS property whose name is made up of **more than one hyphenated word**, join them by **capitalizing the following word**:
 - color** is manipulated as **color**
 - font-size** is manipulated as **fontSize**
 - text-align** is handled as **textAlign**
- Exception: **"float"** property is **styleFloat** in IE and **cssFloat** in others

DHTML: Accessing the CSS properties values

- Only those properties set **inline** or dynamically through javascript can be accessed through the **style** object.

```
<div id='layer1' class='a_style' style='left:1px;color:orange;'>
  Information on the layer
</div>
....
alert(document.getElementById('layer1').style['left'];
```

- The **style** property is **read/write**
- To access values defined in a style sheet:
 - window.getComputedStyle(element, pseudoelm)** is an object that stores the value of each defined CSS property. The pseudoelm parameter is specified if you want to access the properties associated with this selector.
 - ATT: The object returned with **getComputedStyle** is **read-only**

DHTML: Accessing the CSS properties values

```
<style type='text/css'>
.a_style{
    position:absolute;
    top:80px;
    font-size:38px;
}
</style>

<div id='layer1' class='a_style' style='left:150;color:orange;'>
    Information to be shown
</div>
<script language='javascript'>

function CSSValueReturn (element, property){
    let elem = document.getElementById(element);
    let value = window.getComputedStyle(elem)[property];

    //It could also be:
    // let mystyle = window.getComputedStyle(elem);
    // let value = mystyle.getPropertyValue(property);
    return valor;
}

...
// onload
elem = document.getElementById('layer1');
alert(CSSValueReturn ('layer1','fontSize'));
```

DHTML: CSS file and rules update

- You can [change the entire style sheet](#) simply by identifying the link element, accessing this element, and changing the URL assigned to the "href" property:

```
<link href="style00.css" type="text/css" rel="stylesheet" id="style">
document.getElementById('style').href = 'URLNewStyleFile';
```

- Individual rules can be accessed via [rules](#) (IE) and [cssRules](#) (FF and others)

if (IE)

```
myCSSRules = document.styleSheets[0].rules;
```

else

```
myCSSRules = document.styleSheets[0].cssRules;
```

```
// myCSSRules will be an array with the set of rules.
```

```
//Properties of each of the elements in that set of rules:
```

- [cssText](#) - Full text of the rule //not in IE
- [selectorText](#) - Rule selector (label)
- [style](#) - provides access to read/modify the value of a CSS attribute

DHTML: Classes updating

- You can change an element class using `className` property

```
<style type="text/css">
    .class1 {color:red; font-size:16pt;}
    .class2 {color:blue;font-size:24pt;}
</style>
<script type="text/javascript">
    function classChange(name, newclass) {
        document.getElementById(name).className = newclass;
    }
</script>
<input type="button" value='Class change' onclick = "classChange(layer0, 'class2')">
<input type="button" value='Again' onclick = "classChange('layer0', 'class1')">
```

ECMA6: Promises

- Promises are methods that are normally executed **asynchronously** and that cause a certain function to be executed in turn if the asynchronous call is resolved successfully or another function in case of an error.
- It is created with `new Promise()` to which the function to be executed is passed and which, in turn, automatically receives two functions **resolve** and **reject** to use in case of success and failure, respectively.
- When creating the promise, it is indicated with the **then** method the function to execute in case of success and with the **catch** method the function to execute in case of failure.

```
let myPromise = new Promise (function (resolve, reject){
    let time = parseInt(Math.random()*5);
    setTimeout(function(){
        if (time <= 3)
            resolve ('OK');
        else
            reject ('Error');
    }, time*1000);
});

myPromise.then(
    (message)=>{console.log(message)}
).catch(
    (error)=>{console.log(error)}
);
```

async/await

miw.etsisi.upm.es • 131

- A function defined as “**async**” can pause its execution until it receives the result of a **resolve** promise.

```
function resolveAfter2Seconds(x) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

async function add1(x) {
  const a = await resolveAfter2Seconds(20);
  const b = await resolveAfter2Seconds(30);
  return x + a + b;
}

add1(10).then(v => {
  console.log(v); // prints 60 after 4 seconds.
});

async function add2(x) {
  const p_a = resolveAfter2Seconds(20);
  const p_b = resolveAfter2Seconds(30);
  return x + await p_a + await p_b;
}

add2(10).then(v => {
  console.log(v); // prints 60 after 2 seconds.
});
```

async/await

miw.etsisi.upm.es • 132

```
<div id='answer'></div>
<br /><br />
<form>
  <button type='button' id='btn'>Click to process</button>
</form>
<script type='text/javascript'>
  function serviceRequest(){
    let resolvetime = Math.floor( Math.random() * 1500 );
    var myPromise = new Promise(function (resolve, reject){
      setTimeout(function() {
        resolve( tiemposolve + ":" + "<br/> Correcto" );
      }, resolvetime);
    })
    return myPromise;
  }

  document.getElementById('btn').onclick = async function() {
    document.getElementById('answer').innerHTML = "";
    let message = await serviceRequest();
    document.getElementById('answer').innerHTML = message;
  }
</script>
```

Modules in ECMA6: *import and export*

- Modern browsers support the creation and use of modules for better organization of javascript code.
- The value "**module**" must be used for the external file in **type**

```
<script type="module" src="library.js"></script>
```
- Code in modules runs in **strict mode**
- import and export** cannot be used in "scripts", only in modules
- import can only be done in top level modules
- Functions, variables, constants or classes can be exported (and imported)
- They are exported with the **export** keyword:
 - Indicating it in the declaration:


```
export function myfunction(){ ... }
export const myvalue = "Santiago"
```
 - Or at the end of the module:


```
export { myfunction, myvalue };
```

They are imported indicating the route:

```
import { myfunction } from "../modules/mymodule.js"
```

Modules in ECMA6: *import and export(ii)*

- They can be exported and imported by renaming them with the **as** operator:


```
export {
  myfunction as mifuncion,
  myvalue as mivalor
};
import { myfunction as mifuncion } from "mymodule.js"
```
- The namespace can be used for renaming:


```
import * as Module from 'mymodule.js';
```

 And it is used with the identifier:


```
Module.myfunction();
```
- Can be dynamically loaded as a promise

```
import('../modules/mymodule.js')
  .then((module) => {
    // processed with the module.
  })
```

Web Storage

- HTML5 supports persistent browser storage between 5M and 10M
- It is necessary that the browser is "online" to activate these capabilities
- There are two types of web storage:
 - **Local storage** that allows data to be stored and retrieved by any page of a website. The data remains stored by the browser
 - **Session storage** that allows data to be stored and retrieved by the pages of a website as long as the browser tab in which they have been stored is not closed
- The data is managed through the *localStorage* or *sessionStorage* objects depending on whether it is one or the other storage
- The standard is at <http://dev.w3.org/html5/webstorage/>

Web Storage

- The *localStorage* or *sessionStorage* objects have the following methods and properties:
 - *setItem(key,value)* which stores the key/value pair
 - *getItem(key)* which returns the value associated with key
 - *removeItem(key)* which removes the item identified by key
 - *key(n)* which returns the key number corresponding to n
 - *clear* which removes all items
 - *length* which returns the number of pairs stored
- The simplest way to access a local value is through the *associative array format or dot notation*, that is, *localStorage[key]* or *localStorage.key* returns the value associated with key
- The **storage** event is fired on all pages loaded in the browser when a new storage object is created or its content is changed (insertion, modification or deletion).